
Team_Ginkgo

Team Ginkgo

Feb 26, 2025

CONTENTS:

Python Module Index	7
Index	9


```
class config.Config(stream_or_path, **kwargs)
```

Bases: object

```
as_dict()
```

```
close()
```

```
convert_string(s)
```

```
get(key, default=None)
```

```
load(stream)
```

```
load_file(path, encoding=None)
```

```
exception config.ConfigError
```

Bases: ValueError

```
exception config.ConfigFormatError
```

Bases: ParserError

A simple in-process python *scheduler* library for *datetime* objects.

Author: Jendrik A. Potyka, Fabian A. Preiss

```
class scheduler.Scheduler(*, max_exec: int = 0, tzinfo: ~datetime.tzinfo | None = None, priority_function:
    ~typing.Callable[[float, ~scheduler.threading.job.Job, int, int], float] = <function
    linear_priority_function>, jobs:
    ~collections.abc.Iterable[~scheduler.threading.job.Job] | None = None, n_threads:
    int = 1, logger: ~logging.Logger | None = None)
```

Bases: BaseScheduler[Job, Callable[[...], None]]

Implementation of a scheduler for callback functions.

This implementation enables the planning of **|Job|**s depending on time cycles, fixed times, weekdays, dates, offsets, execution counts and weights.

Notes

Due to the support of *datetime* objects, *scheduler* is able to work with timezones.

Parameters

- **tzinfo** (*datetime.tzinfo*) – Set the timezone of the **|Scheduler|**.
- **max_exec** (*int*) – Limits the number of overdue **|Job|**s that can be executed by calling function *Scheduler.exec_jobs()*.
- **priority_function** (*Callable[[float, Job, int, int], float]*) – A function handle to compute the priority of a **|Job|** depending on the time it is overdue and its respective weight. Defaults to a linear priority function.
- **jobs** (*set[Job]*) – A collection of job instances.
- **n_threads** (*int*) – The number of worker threads. 0 for unlimited, default 1.
- **logger** (*Optional[logging.Logger]*) – A custom Logger instance.

```
cyclic(timing: timedelta, handle: Callable[[...], None], **kwargs) → Job
```

Schedule a cyclic *Job*.

Use a *datetime.timedelta* object or a *list* of *datetime.timedelta* objects to schedule a cyclic **|Job|**.

Parameters

- **timing** (*TimingTypeCyclic*) – Desired execution time.
- **handle** (*Callable[... , None]*) – Handle to a callback function.
- ****kwargs** – **|Job|** properties, optional

kwargs are used to specify **|Job|** properties.

Here is a list of available **|Job|** properties:

Returns

Instance of a scheduled **|Job|**.

Return type

Job

daily(*timing: time | list[time], handle: Callable[[...], None], **kwargs*) → Job

Schedule a daily *Job*.

Use a *datetime.time* object or a *list* of *datetime.time* objects to schedule a **|Job|** every day.

Parameters

- **timing** (*TimingDailyUnion*) – Desired execution time(s).
- **handle** (*Callable[... , None]*) – Handle to a callback function.
- ****kwargs** – **|Job|** properties, optional

kwargs are used to specify **|Job|** properties.

Here is a list of available **|Job|** properties:

Returns

Instance of a scheduled **|Job|**.

Return type

Job

delete_job(*job: Job*) → None

Delete a *Job* from the *Scheduler*.

Parameters

job (*Job*) – **|Job|** instance to delete.

Raises

SchedulerError – Raises if the **|Job|** of the argument is not scheduled.

delete_jobs(*tags: set[str] | None = None, any_tag: bool = False*) → int

Delete a set of **|Job|**s from the **|Scheduler|** by tags.

If no tags or an empty set of tags are given defaults to the deletion of all **|Job|**s.

Parameters

- **tags** (*Optional[set[str]]*) – Set of tags to identify target **|Job|**s.
- **any_tag** (*bool*) – False: To delete a **|Job|** all tags have to match. True: To delete a **|Job|** at least one tag has to match.

exec_jobs(*force_exec_all*: *bool* = *False*) → int

Execute scheduled *Jobs*.

By default executes the **|Job|**s that are overdue.

|Job|s are executed in order of their priority examples.weights. If the **|Scheduler|** instance has a limit on the job execution counts per call of `exec_jobs()`, via the *max_exec* argument, **|Job|**s of lower priority might not get executed when competing **|Job|**s are overdue.

Parameters

force_exec_all (*bool*) – Ignore the both - the status of the **|Job|** timers as well as the execution limit of the **|Scheduler|**

Returns

Number of executed **|Job|**s.

Return type

int

get_jobs(*tags*: *set[str] | None* = *None*, *any_tag*: *bool* = *False*) → set[Job]

Get a set of **|Job|**s from the **|Scheduler|** by tags.

If no tags or an empty set of tags are given defaults to returning all **|Job|**s.

Parameters

- **tags** (*set[str]*) – Tags to filter scheduled **|Job|**s. If no tags are given all **|Job|**s are returned.
- **any_tag** (*bool*) – False: To match a **|Job|** all tags have to match. True: To match a **|Job|** at least one tag has to match.

Returns

Currently scheduled **|Job|**s.

Return type

set[Job]

hourly(*timing*: *time | list[time]*, *handle*: *Callable[...]*, *None*], ***kwargs*) → Job

Schedule an hourly *Job*.

Use a *datetime.time* object or a *list* of *datetime.time* objects to schedule a **|Job|** every hour.

Notes

If given a *datetime.time* object with a non zero hour property, this information will be ignored.

Parameters

- **timing** (*TimingDailyUnion*) – Desired execution time(s).
- **handle** (*Callable[...]*, *None*) – Handle to a callback function.
- ****kwargs** – **|Job|** properties, optional
kwargs are used to specify **|Job|** properties.
 Here is a list of available **|Job|** properties:

Returns

Instance of a scheduled **|Job|**.

Return type

Job

property jobs: set[Job]

Get the set of all *Jobs*.

Returns

Currently scheduled **|Job|**s.

Return type

set[Job]

minutely(*timing: time | list[time], handle: Callable[[...], None], **kwargs*) → Job

Schedule a minutely *Job*.

Use a *datetime.time* object or a *list* of *datetime.time* objects to schedule a **|Job|** every minute.

Notes

If given a *datetime.time* object with a non zero hour or minute property, these information will be ignored.

Parameters

- **timing** (*TimingDailyUnion*) – Desired execution time(s).
- **handle** (*Callable[... , None]*) – Handle to a callback function.
- ****kwargs** – **|Job|** properties, optional

kwargs are used to specify **|Job|** properties.

Here is a list of available **|Job|** properties:

Returns

Instance of a scheduled **|Job|**.

Return type

Job

once(*timing: datetime | timedelta | Weekday | time, handle: Callable[[...], None], * (Keyword-only parameters separator (PEP 3102)), args: tuple[Any, ...] | None = None, kwargs: dict[str, Any] | None = None, tags: Iterable[str] | None = None, alias: str | None = None, weight: float = 1*) → Job

Schedule a oneshot *Job*.

Parameters

- **timing** (*TimingOnceUnion*) – Desired execution time.
- **handle** (*Callable[... , None]*) – Handle to a callback function.
- **args** (*tuple[Any]*) – Positional argument payload for the function handle within a **|Job|**.
- **kwargs** (*Optional[dict[str, Any]]*) – Keyword arguments payload for the function handle within a **|Job|**.
- **tags** (*Optional[Iterable[str]]*) – The tags of the **|Job|**.
- **alias** (*Optional[str]*) – Overwrites the function handle name in the string representation.
- **weight** (*float*) – Relative weight against other **|Job|**s.

Returns

Instance of a scheduled **|Job|**.

Return type

Job

weekly(*timing*: Weekday | list[Weekday], *handle*: Callable[[...], None], ***kwargs*) → Job

Schedule a weekly *Job*.

Use a *tuple* of a *Weekday* and a *datetime.time* object to define a weekly recurring **|Job|**. Combine multiple desired *tuples* in a *list*. If the planed execution time is *00:00* the *datetime.time* object can be ignored, just pass a *Weekday* without a *tuple*.

Parameters

- **timing** (*TimingWeeklyUnion*) – Desired execution time(s).
- **handle** (*Callable[... , None]*) – Handle to a callback function.
- ****kwargs** – **|Job|** properties, optional

kwargs are used to specify **|Job|** properties.

Here is a list of available **|Job|** properties:

Returns

Instance of a scheduled **|Job|**.

Return type

Job

exception scheduler.SchedulerError

Bases: Exception

Generic Scheduler exception.

PYTHON MODULE INDEX

C

`config`, 1

S

`scheduler`, 1

INDEX

A

`as_dict()` (*config.Config method*), 1

C

`close()` (*config.Config method*), 1

`config`

 module, 1

`Config` (*class in config*), 1

`ConfigError`, 1

`ConfigFormatError`, 1

`convert_string()` (*config.Config method*), 1

`cyclic()` (*scheduler.Scheduler method*), 1

D

`daily()` (*scheduler.Scheduler method*), 2

`delete_job()` (*scheduler.Scheduler method*), 2

`delete_jobs()` (*scheduler.Scheduler method*), 2

E

`exec_jobs()` (*scheduler.Scheduler method*), 2

G

`get()` (*config.Config method*), 1

`get_jobs()` (*scheduler.Scheduler method*), 3

H

`hourly()` (*scheduler.Scheduler method*), 3

J

`jobs` (*scheduler.Scheduler property*), 3

L

`load()` (*config.Config method*), 1

`load_file()` (*config.Config method*), 1

M

`minutely()` (*scheduler.Scheduler method*), 4

`module`

 config, 1

 scheduler, 1

O

`once()` (*scheduler.Scheduler method*), 4

S

`scheduler`

 module, 1

`Scheduler` (*class in scheduler*), 1

`SchedulerError`, 5

W

`weekly()` (*scheduler.Scheduler method*), 4